Smart Farm IoT

Mini-projet — Simulation logicielle d'une ferme connectée sécurisée

Louis VANLAER, Ziyad MAKROUM Cyber 3C

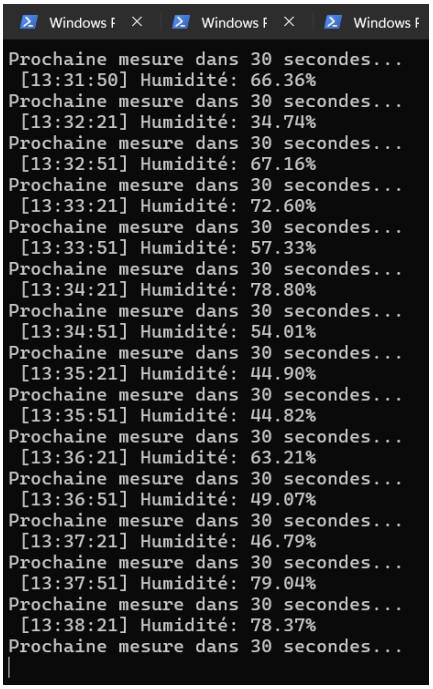
Introduction

Le but du projet est de simuler une ferme connectée en Python : deux capteurs (température et humidité), un contrôleur qui applique des règles, et deux actionneurs (ventilation et irrigation). Tout communique en MQTT via un broker Mosquitto. On part d'un système qui marche mais qui n'a aucune sécurité, puis on le sécurise petit à petit.

Étape	Ce qu'on ajoute	Principe visé
Jour 1	Capteurs, contrôleur, actionneurs	— (aucune sécurité)
Jour 2	Tokens + chiffrement AES	Authentification, confidentialité
Jour 3	Anti-replay, ACL, monitoring	Intégrité, contrôle d'accès, disponibilité

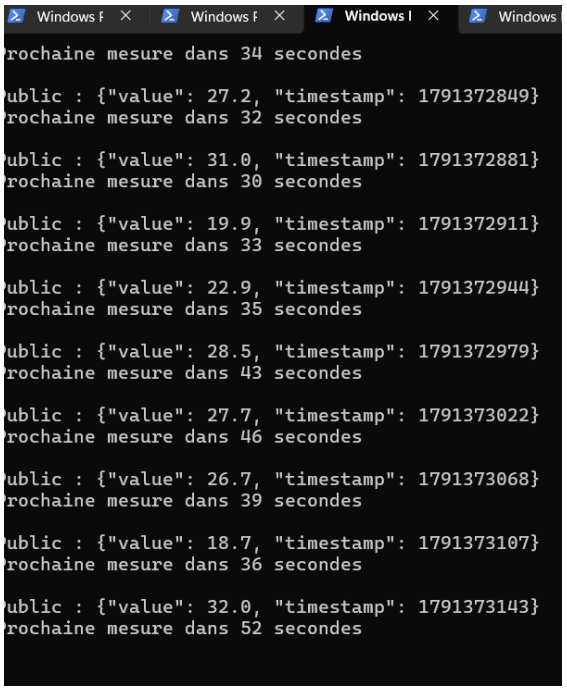
Jour 1 — Système IoT fonctionnel (sans sécurité)

1.1 Capteurs



Capture 1 — Capteur d'humidité (humidity.py)

Le script `humidity.py` génère une mesure d'humidité toutes les 30 secondes et la publie sur le topic `/farm/humidity`.



Capture 2 — Capteur de température (temperature.py)

Le script `temperature.py` publie la température sur `/farm/temperature` avec un intervalle aléatoire entre 30 et 60 secondes. Les messages sont au format JSON avec la valeur et le timestamp, et ils sont envoyés en clair.

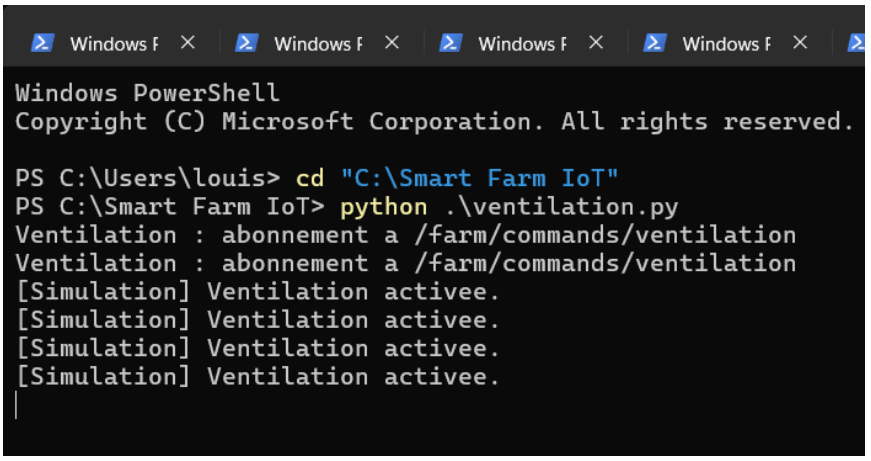
1.2 Contrôleur

```
PS C:\Smart Farm IoT> python controller.py
Controller subscribed to temperature and humidity.
Command queued: /farm/commands/ventilation {"command": "ON", "timestamp": 1791372543}
Command queued: /farm/commands/irrigation {"command": "ON", "timestamp": 1791372560}
Command queued: /farm/commands/ventilation {"command": "ON", "timestamp": 1791372600}
Command queued: /farm/commands/irrigation {"command": "ON", "timestamp": 1791372650}
Command queued: /farm/commands/ventilation {"command": "ON", "timestamp": 1791372701}
Command queued: /farm/commands/irrigation {"command": "ON", "timestamp": 1791372741}
Command queued: /farm/commands/ventilation {"command": "ON", "timestamp": 1791372881}
Command queued: /farm/commands/ventilation {"command": "ON", "timestamp": 1791373143}
```

Capture 3 — Contrôleur : commandes envoyées aux actionneurs

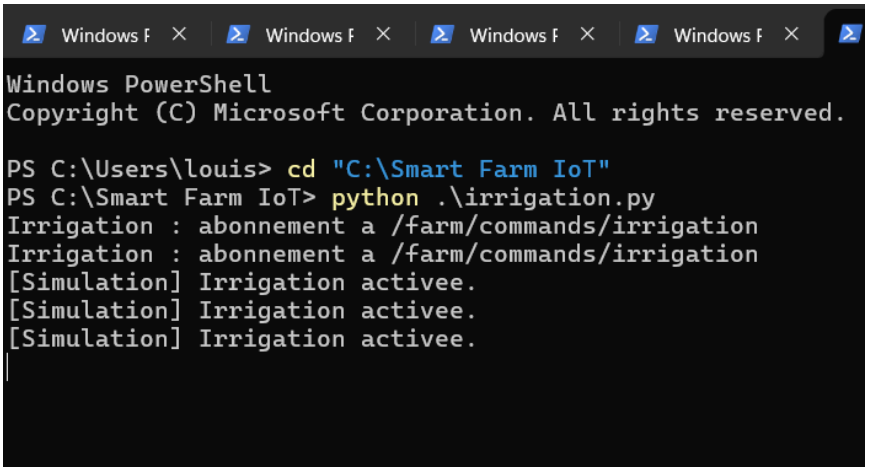
Le contrôleur est abonné aux deux topics des capteurs. Chaque ligne « Command queued » correspond à une règle qui s'est déclenchée : soit la température a dépassé 30 °C (commande ON pour la ventilation), soit l'humidité est passée sous 40 % (commande ON pour l'irrigation).

1.3 Actionneurs



Capture 4 — Actionneur de ventilation (ventilation.py)

On lance `ventilation.py`, qui s'abonne au topic `/farm/commands/ventilation`. À chaque commande envoyée par le contrôleur, la ventilation s'active, ce qu'on simule par un simple affichage dans la console.

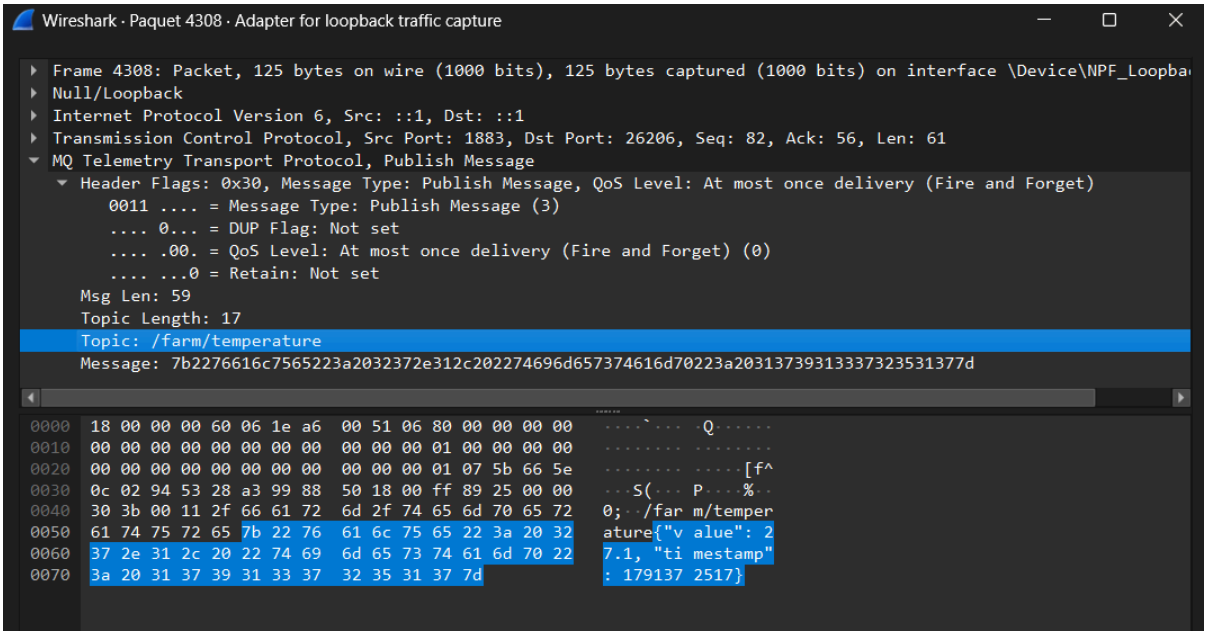


Capture 5 — Actionneur d'irrigation (irrigation.py)

Même principe pour `irrigation.py`, qui écoute `/farm/commands/irrigation`. Quand l'humidité passe sous 40 %, le contrôleur envoie la commande et l'irrigation s'active dans la console.

Jour 2 — Analyse et premières sécurisations

2.1 Analyse du système non sécurisé



Capture 6 — Capture Wireshark d'un message MQTT en clair

On a lancé Wireshark sur l'interface loopback pendant que les capteurs tournaient. On voit bien le paquet MQTT envoyé sur le topic `/farm/temperature`, et dans la partie hexadécimale on peut lire directement la température (27.1) et le timestamp. Le message n'est pas chiffré, donc n'importe qui qui écoute le réseau peut lire les données. Rien n'empêche non plus de modifier le message ou de le renvoyer plus tard (replay). Il n'y a aucun token non plus, donc on ne peut pas savoir qui a vraiment envoyé le message.

2.2 Authentification par tokens

```
PS C:\Smart Farm IoT> python controller.py
Controller subscribed to temperature and humidity.
[OK] Token valide pour /farm/temperature
[OK] Token valide pour /farm/humidity
Command queued: /farm/commands/irrigation {"command": "ON", "timestamp": 1791377528, "token": "CTRL_9a56c4f8"}
[OK] Token valide pour /farm/humidity
[OK] Token valide pour /farm/temperature
Command queued: /farm/commands/ventilation {"command": "ON", "timestamp": 1791377572, "token": "CTRL_9a56c4f8"}
[OK] Token valide pour /farm/humidity
[OK] Token valide pour /farm/humidity
[OK] Token valide pour /farm/temperature
Command queued: /farm/commands/ventilation {"command": "ON", "timestamp": 1791377622, "token": "CTRL_9a56c4f8"}
[OK] Token valide pour /farm/humidity
[OK] Token valide pour /farm/temperature
[OK] Token valide pour /farm/humidity
Command queued: /farm/commands/irrigation {"command": "ON", "timestamp": 1791377678, "token": "CTRL_9a56c4f8"}
[OK] Token valide pour /farm/temperature
Command queued: /farm/commands/ventilation {"command": "ON", "timestamp": 1791377706, "token": "CTRL_9a56c4f8"}
[OK] Token valide pour /farm/humidity
[OK] Token valide pour /farm/temperature
[OK] Token valide pour /farm/humidity
[OK] Token valide pour /farm/humidity
[OK] Token valide pour /farm/temperature
Command queued: /farm/commands/ventilation {"command": "ON", "timestamp": 1791377769, "token": "CTRL_9a56c4f8"}
[OK] Token valide pour /farm/humidity
[OK] Token valide pour /farm/temperature
```

Capture 7 — Contrôleur : tokens valides

Chaque capteur a maintenant son propre token, rangé dans `tokens.json`, et l'envoie dans chaque message. Le contrôleur vérifie le token avant de traiter la mesure : ici tous les

messages sont acceptés (« [OK] Token valide »). Les commandes envoyées par le contrôleur contiennent elles aussi son token.



Capture 8 — Contrôleur : rejet d'un token invalide

Pour tester, on a remplacé `TOKEN = TOKENS["humidity"]` par `TOKEN = "FAUX_TOKEN"` dans le capteur d'humidité. Le contrôleur rejette alors tous ses messages avec « [SECURITE] Token invalide », alors que ceux du capteur de température continuent d'être acceptés.

Par contre, à ce stade le token est encore envoyé en clair dans MQTT. Quelqu'un qui capture le trafic avec Wireshark pourrait donc le récupérer et l'utiliser. C'est pour ça que l'étape suivante ajoute le chiffrement AES.

2.3 Chiffrement des messages (AES)



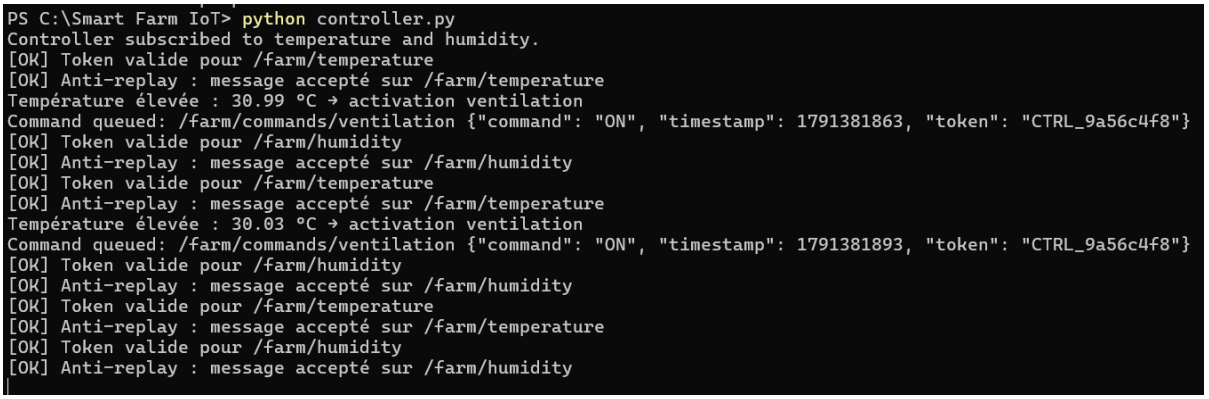
Capture 9 — Capteur de température : messages chiffrés en AES

Le message JSON (valeur, timestamp et token) est maintenant chiffré avec la clé AES partagée (`aes.key`), puis encodé en Base64 avant d'être publié. Dans la console, on voit que chaque mesure donne une suite de caractères illisible. Quelqu'un qui écoute le réseau ne

voit plus la température ni le token : seul le contrôleur, qui a la clé, peut déchiffrer le message et appliquer les règles.

Jour 3 — Sécurisation progressive

3.1 Protection contre le replay

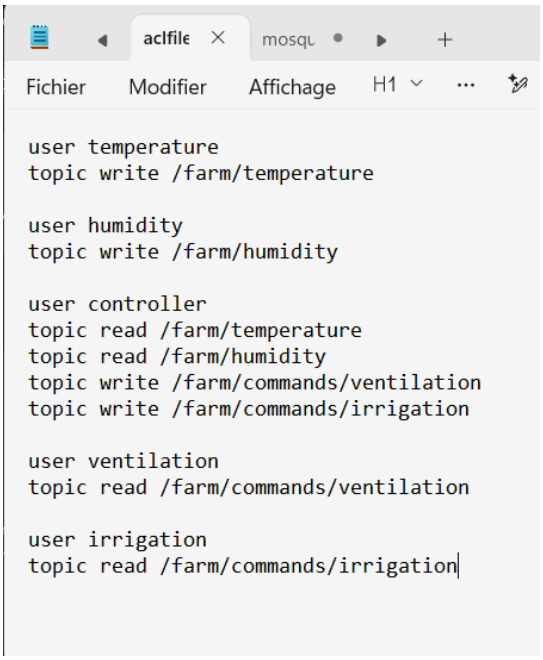


Capture 10 — Contrôleur : vérification anti-replay

Chaque message contient maintenant un timestamp et un nonce (identifiant aléatoire unique). Après avoir vérifié le token, le contrôleur contrôle que le message n'est pas trop ancien et que son nonce n'a jamais été reçu. Si c'est bon, il affiche « [OK] Anti-replay : message accepté » et applique les règles, comme ici avec l'activation de la ventilation à 30,99 °C. Un message rejoué à l'identique serait rejeté, puisque son nonce est déjà connu.

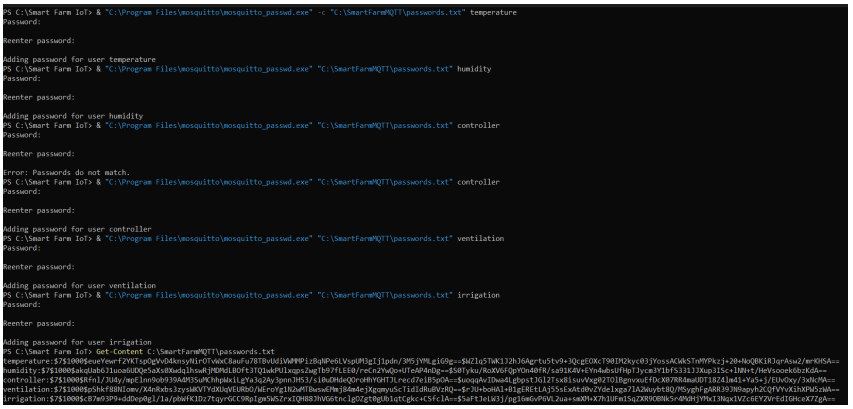
Ces champs sont à l'intérieur du message chiffré : un attaquant ne peut donc pas changer le timestamp ou mettre un nouveau nonce pour faire passer un vieux message.

3.2 Contrôle d'accès MQTT (ACL)





On a désactivé les connexions anonymes dans `mosquitto.conf` (`allow_anonymous false`) et ajouté un fichier de mots de passe et un fichier d'ACL. Dans l'`aclfile`, chaque dispositif a seulement les droits dont il a besoin : les capteurs publient sur leur topic, les actionneurs lisent leurs commandes, et le contrôleur lit les capteurs et publie les commandes.



3.3 Monitoring et alertes



Une fois le broker relancé avec les ACL, on a vérifié que le système marche toujours. Le contrôleur se connecte avec son compte, reçoit bien les mesures des deux capteurs (token valide, anti-replay OK) et envoie des commandes chiffrées à la ventilation et à l'irrigation. Les ACL ne bloquent donc pas les dispositifs légitimes.

Cette capture montre que tout fonctionne, mais pas encore qu'un intrus est bloqué. Pour compléter le 3.2, une capture d'une connexion refusée serait idéale. Par exemple :

mosquitto_sub -h localhost -t "#" -v

Sans identifiants, elle doit donner « Connection Refused: not authorised